NEXO SOCIAL

Panel de administración

Catálogo de capacidades

Todo lo que el panel *podría* tener, con una señal de esfuerzo y de valor para la demo en cada capacidad. Es un catálogo para elegir, no un plan de trabajo: construir todo esto es varios meses, y la sección 7 dice qué dejaría afuera del hackathon y por qué.

Alcance	Consola interna de Operador y Moderador
Base verificada	Scopes y auditoría ya existen en Serverpod; el dominio no
Capacidades	92 capacidades en 14 módulos
Fecha	28 de septiembre de 2026

01

Para qué sirve este documento

No propone alcance nuevo: propone construir alcance que ya está escrito.

El Notion del proyecto ya define al **Operador** —"revisar reportes, suspender contenido y cuentas, auditar"— y dice, con esas palabras, que *"Operador tiene consola separada y auditada"*. El panel no es una idea nueva; es una pieza especificada y no construida.

También encaja con el objetivo declarado del hackathon: *"La demo debe probar que no es una maqueta: permisos y estados se aplican en backend."* Toda otra pantalla del producto se puede fingir con mocks. Una consola que le quita a alguien la capacidad de emitir, no: o el servidor lo rechaza, o no lo rechaza.

Las capacidades están agrupadas por módulo y cada una lleva dos señales:

- **Esfuerzo** — **S** son horas, **M** es un día, **L** son varios días o depende de terceros.
- **Demo** — cuánto rinde *en la presentación*, que no es lo mismo que cuánto vale el producto. Una cola de apelaciones es imprescindible en producción y no se ve en una demo.

> **Cómo leer la columna "Demo"**
>
> Alto significa que produce un momento verificable delante del jurado: algo cambia en una pantalla porque el servidor lo decidió. Bajo no significa inútil, significa invisible en noventa segundos.

02

Lo que ya existe, verificado en el repo

Conviene partir de lo que hay y no de lo que se supone que hay.

En el backend (Serverpod)

- **Scopes.** `NexoScopes.moderator` y `NexoScopes.admin` en `shared/auth/auth_context.dart`, sobre el usuario de Serverpod. El comentario del archivo aclara algo importante: *"creator no es un scope: vive en el perfil"*.
- **Auditoría transaccional.** `AuditService.record()` acepta una `Transaction`, así que el registro se confirma o se descarta *junto con* la acción auditada. Eso es lo que impide que la auditoría mienta cuando una operación falla a medias.
- **Tabla inmutable.** `audit_log` con actor, acción, entidad, metadata y fecha; índices por entidad y por actor.
- **Interfaces de gateway** para media en vivo y almacenamiento, todavía sin implementación.

En el cliente (Flutter)

- Feature `moderation/` completa en mock: cola de reportes con severidad, acciones de ocultar, silenciar con duración, expulsar y reportar, y registro de auditoría append-only.
- Hoja de moderación contextual que *devuelve una decisión* en vez de aplicarla, así que sirve igual en la sala de un vivo y en la consola.
- Shell responsive con `/moderation` y `/studio` como destinos sólo de escritorio.

> **Dos huecos que condicionan todo lo demás**

El dominio del backend está vacío: no hay modelos de identidad, contenido, social, live ni moderación, sólo los compartidos. Y en Flutter, `UserRole` es visitor / user / creator / moderator, sin operador: el enum del cliente y los scopes del servidor todavía no coinciden.

03

Dos decisiones que condicionan el resto

Es una consola, no un dashboard

Un dashboard muestra números. Una consola *hace* cosas. Los criterios de aceptación del Notion son todos verbos: "el usuario no autorizado no obtiene token broadcaster", "el comentario oculto desaparece del feed y conserva auditoría". Ningún criterio pide un gráfico.

La consecuencia práctica es de orden: **ninguna métrica antes de que las acciones funcionen**. Un panel con analítica sobre datos inventados resta credibilidad en vez de sumarla, porque el jurado sabe leer un número que nadie generó.

El permiso se aplica en el servidor

Esconder la ruta en Flutter no es seguridad: es una decisión de layout. La consola tiene que rechazar desde el backend aunque alguien escriba `/admin` a mano, y esa diferencia es demostrable en vivo.

El guard del router ya existe y está aislado como función pura (`guardRedirect`), así que sumar un rol es barato. Lo que no es barato —y es lo que de verdad importa— es que cada endpoint verifique el scope.

04

Modelo de permisos

Todo el catálogo cuelga de esta tabla: sin roles claros, cada capacidad hereda la ambigüedad.

Rol	Dónde vive	Alcance	Puede
Usuario	Sin scope	Su propio contenido	Consumir, reaccionar, comentar, reportar
Creador	Perfil, no scope	Su comunidad	Publicar, programar e iniciar lives, crear membresías
Host	Por sesión	Un live propio	Invitar cohosts, iniciar y finalizar grabación
Moderador	`moderator`	La comunidad asignada	Ocultar, silenciar, expulsar, reportar
Operador	`admin`	Toda la plataforma	Suspender cuentas y contenido, verificar, auditar

El detalle que se pasa por alto

Moderador y Operador no son el mismo rol con distinto tamaño. El moderador actúa dentro de una comunidad que le fue asignada; el operador actúa sobre la plataforma y toda su actividad queda auditada. Si se colapsan en uno solo, la auditoría deja de poder responder quién tenía derecho a hacer qué.

05

Catálogo por módulo

92 capacidades en 14 módulos. Ninguna lista pretende ser obligatoria.

A · Resumen operativo

La pantalla de entrada. Su trabajo es responder "qué necesita atención ahora", no "cómo viene el mes".

Capacidad	Esf.	Demo	Requiere
Reportes pendientes, con desglose por severidad	S	Alto	Módulo moderation
Lives emitiendo en este momento	S	Alto	Módulo live
Acciones de moderación en las últimas 24 h	S	Medio	audit_log
Cuentas creadas y suspendidas hoy	S	Bajo	Módulo identity
Cola de verificación pendiente	S	Medio	Perfil de creador
Estado de los gateways: media, live, pagos	M	Bajo	Gateways reales
Incidentes abiertos y quién los tomó	M	Bajo	Modelo de caso

B · Moderación y Trust & Safety

El corazón del panel, y lo único que el Notion marca como P0 desde el principio.

Capacidad	Esf.	Demo	Requiere
Cola de reportes con filtros por severidad, tipo y antigüedad	S	Alto	Ya existe en mock
Detalle del caso con la evidencia original preservada	M	Alto	Soft delete
Resolver en un paso: aplicar acción y cerrar el reporte	S	Alto	Ya existe en mock
Ocultar comentario o publicación para toda la audiencia	M	Alto	Módulo content
Silenciar usuario con duración acotada (15 min, 1 h, 24 h)	S	Alto	Ya existe en mock
Expulsar de un live y revocar su token de emisión	M	Alto	Módulo live
Bloquear usuario de un canal o comunidad	M	Medio	Módulo social
Descartar reporte como infundado	S	Medio	Módulo moderation
Agrupar N reportes sobre el mismo contenido en un caso	M	Medio	Modelo de caso
Historial de sanciones por usuario (reincidencia)	M	Alto	audit_log + consulta
Motivos tipificados en vez de texto libre	S	Medio	Enum compartido
Notas internas del caso, visibles sólo para el equipo	S	Bajo	Modelo de caso
Escalar un caso a otro operador	M	Bajo	Asignación
Cola de apelaciones	L	Bajo	Flujo de apelación
Listas de palabras y patrones bloqueados	M	Bajo	Módulo moderation
Revisión previa a publicar para cuentas nuevas	L	Bajo	Estado de publicación

C · Cuentas y usuarios

Donde aterrizan la mayoría de los casos, y donde vive la idea de "banear por correo".

Capacidad	Esf.	Demo	Requiere
Búsqueda por correo, usuario o id	S	Alto	Módulo identity
Ficha de cuenta: estado, rol, alta, sanciones	M	Alto	Módulo identity
Suspender y restaurar cuenta	M	Alto	Estado de cuenta
Baneo permanente	M	Alto	Estado de cuenta
Revocar sesiones activas (echar al usuario ya logueado)	M	Alto	Auth de Serverpod
Cambiar rol: usuario, creador, moderador, operador	S	Alto	Scopes
Bloquear registro por dominio de correo	M	Medio	Módulo identity
Forzar reverificación de correo	M	Bajo	Email IdP
Ver como (solo lectura, con auditoría obligatoria)	L	Medio	Impersonación
Exportar los datos de un usuario	L	Bajo	Cumplimiento
Eliminar cuenta y anonimizar su rastro	L	Bajo	Cumplimiento

> **Un ban que no se ve no demuestra nada**
>
> Bloquear futuros inicios de sesión es invisible en una demo: nadie inicia sesión dos veces delante del jurado. Lo que se ve es la **revocación de sesión** — el usuario baneado queda fuera en ese mismo momento. Y una nota de modelo: se banea la **cuenta** y se **busca** por correo. El correo es el identificador humano, no la clave.

D · Creadores y verificación

El módulo que convierte la verificación en una capacidad y no en una insignia.

Capacidad	Esf.	Demo	Requiere
Cola de solicitudes de verificación	M	Alto	Perfil de creador
Aprobar o rechazar con motivo	S	Alto	Perfil de creador
Revocar verificación ya otorgada	S	Alto	Perfil de creador
Gate: sólo verificados pueden iniciar lives	M	Alto	Módulo live
Gate: sólo verificados pueden cobrar membresías	M	Medio	Módulo billing
Niveles: verificado, partner, mentor	M	Medio	Perfil de creador
Requisitos automáticos: antigüedad, seguidores, sin sanciones	L	Bajo	Módulo social
Límites por creador: lives simultáneos, duración máxima	M	Bajo	Módulo live

E · Contenido

Capacidad	Esf.	Demo	Requiere
Búsqueda de publicaciones por autor, hashtag o fecha	M	Medio	Módulo content
Ocultar y restaurar una publicación	M	Alto	Soft delete
Marcar como sensible: difuminado y advertencia	M	Medio	Módulo content

Capacidad	Esf.	Demo	Requiere
Quitar de tendencias sin ocultar del perfil	M	Bajo	Ranking
Destacar contenido en el feed	M	Medio	Feed
Gestión de hashtags: bloquear, fusionar, destacar	M	Bajo	Módulo content
Cola de media pendiente de revisión	L	Bajo	Storage gateway

F · Lives

El módulo más vistoso, y el que más depende de que el ciclo de vida exista en el backend.

Capacidad	Esf.	Demo	Requiere
Emisiones activas en tiempo real	M	Alto	Módulo live
Forzar el fin de una emisión	M	Alto	Máquina de estados
Entrar como observador silencioso	L	Alto	Agora
Revocar el token de broadcaster de un cohost	M	Alto	Tokens de Serverpod
Silenciar el chat de un live	S	Medio	Módulo live
Historial de lives y replays	M	Bajo	Módulo live
Despublicar un replay	S	Medio	Estado published
Límites: duración, espectadores, calidad	M	Bajo	Configuración

G · Monetización

Todo esto es P1 con gateway falso, según el propio Notion.

Capacidad	Esf.	Demo	Requiere
Membresías activas por creador	M	Bajo	Módulo billing
Congelar pagos de un creador bajo investigación	M	Medio	Módulo billing
Reembolsos y disputas	L	Bajo	Pasarela real
Cola de payouts	L	Bajo	Pasarela real
Comisión de la plataforma por creador	M	Bajo	Módulo billing

H · Comunidades

Capacidad	Esf.	Demo	Requiere
Listado con estado y persona responsable	M	Medio	Módulo social
Asignar y revocar moderadores	S	Alto	Scopes por comunidad
Archivar o cerrar una comunidad	M	Medio	Módulo social
Reglas propias por comunidad	L	Bajo	Módulo social

I · Comunicación

Capacidad	Esf.	Demo	Requiere
Enviar un aviso in-app a un usuario	M	Medio	Notificaciones
Anuncio global o banner de plataforma	M	Medio	Notificaciones
Notificar a un segmento	L	Bajo	Segmentación
Plantillas de notificación de sanción	S	Medio	Notificaciones

J · Auditoría y cumplimiento

Lo que convierte al panel en defendible. Ya tiene la mitad construida.

Capacidad	Esf.	Demo	Requiere
Historial filtrable por actor, acción, entidad y fecha	M	Alto	Endpoint con DTO
Vista por caso: todo lo ocurrido con un reporte	M	Alto	Índice por entidad
Alertas ante acciones sensibles	M	Bajo	Notificaciones
Exportar la auditoría	S	Bajo	Endpoint
Retención y borrado programado	L	Bajo	Cumplimiento

AuditLog es serverOnly

El modelo está declarado `serverOnly: true`, así que **no se puede leer desde el cliente generado**. Hace falta un endpoint que devuelva un DTO propio. Eso es bueno: obliga a decidir qué puede ver un operador. Pero a las dos de la mañana parece un bug del generador.

K · Configuración de plataforma

Capacidad	Esf.	Demo	Requiere
Feature flags por entorno	M	Medio	Configuración
Límites globales: rate limits, tamaño de archivo	M	Bajo	Configuración
Modo mantenimiento	S	Medio	Configuración
Textos legales versionados	M	Bajo	Módulo content
Conmutar gateways entre mock y real	S	Medio	Gateways

L · Soporte interno

Capacidad	Esf.	Demo	Requiere
Buscar por id de live, publicación o transacción	S	Medio	Varios módulos
Diagnóstico: qué ve un usuario y por qué	L	Alto	Motor de permisos
Bandeja de tickets	L	Bajo	Módulo nuevo

M · IA y automatización

P2 en el Notion, detrás de una interfaz. Nada de esto es necesario para la demo.

Capacidad	Esf.	Demo	Requiere
Clasificación automática de reportes	L	Bajo	Módulo ai
Detección de spam y contenido sensible	L	Bajo	Módulo ai
Sugerencia de acción con nivel de confianza	L	Medio	Módulo ai
Cola de revisión humana de lo que marcó la IA	M	Medio	Módulo ai
Precisión del clasificador y falsos positivos	L	Bajo	Métricas

N · Observabilidad

Capacidad	Esf.	Demo	Requiere
Salud de los servicios	S	Bajo	Health endpoint
Errores recientes agrupados	M	Bajo	Telemetría
Latencia de endpoints críticos	M	Bajo	Telemetría
Consumo de los gateways	M	Bajo	Gateways reales

06

Si hubiera que elegir

Las diez capacidades con mejor relación entre lo que cuestan y lo que prueban.

#	Capacidad	Por qué está acá
1	Rol de operador alineado con `NexoScopes.admin`	Sin esto ninguna otra capacidad se puede proteger de verdad
2	Guard de ruta por scope real, no por esconder la ruta	Es la diferencia entre una consola y una pantalla con suerte
3	Endpoint de auditoría con DTO propio	Obligatorio: el modelo es serverOnly y no viaja al cliente
4	Resolver un reporte con acción y auditoría	Ya existe en mock; conectarlo es de las tareas más baratas
5	Búsqueda de cuentas por correo o usuario	Es la puerta de entrada a casi todo el resto
6	Suspender cuenta y revocar la sesión activa	Un ban sin revocación no se ve; con revocación es el momento
7	Verificar y desverificar creador	Convierte la verificación en una capacidad, no una insignia
8	Gate de lives por verificación	Prueba la tesis entera: el servidor rechaza, no la UI esconde
9	Estado de cuenta visible en el perfil público	Una sanción que no se ve desde afuera no se puede demostrar
10	Forzar el fin de un live en curso	El más espectacular, si el ciclo de vida ya existe en backend

> **Y si sólo entran tres**
>
> Los números 4, 6 y 8: resolución auditada, suspensión con revocación de sesión, y el gate de lives. Los tres se encadenan en un solo guion corto y ninguno se puede fingir desde el cliente.

07

Lo que dejaría afuera del hackathon

Descartar explícitamente cuesta menos que descubrir a mitad de camino que no llega.

Gráficos y analítica

Sin datos reales son decoración, y compiten por el tiempo de lo que sí prueba algo. Un jurado que reconoce un número inventado descuenta credibilidad al resto de la demo.

Apelaciones, acciones masivas y exportación

Necesidades reales de Trust & Safety en producción; cero momento de demostración. Una apelación tarda días en resolverse, que es exactamente lo contrario de lo que se puede mostrar en vivo.

IA y clasificación automática

El Notion ya lo pone en P2 detrás de una interfaz. Un clasificador entrenado con nada clasifica mal, y un clasificador con reglas escritas a mano es una lista de palabras con otro nombre.

Una aplicación separada

El shell responsive ya tiene destinos sólo de escritorio y un panel lateral a partir de 1100 px. Montar la consola ahí cuesta una fracción y ya funciona en macOS y en web. Un segundo proyecto duplica autenticación, tema, routing y despliegue.

El error más caro sería el orden

Construir el resumen con métricas antes que las acciones. El resumen es la primera pantalla que se ve y la última que conviene construir: hasta que las acciones no existen, no hay nada real que resumir.

08

Guion de demostración sugerido

Noventa segundos, y todo lo que ocurre lo decide el servidor.

Paso	Qué se hace	Qué prueba
1	Elena inicia un live desde la app	El camino feliz funciona
2	Un espectador escribe un comentario abusivo y otro lo reporta	El reporte llega a la cola
3	En la consola aparece el reporte con su severidad	El backend es la fuente, no la pantalla
4	El operador lo resuelve: oculta y silencia 24 h	El comentario desaparece para todos
5	Se muestra la auditoría con actor, acción y hora	Nada se borra; queda evidencia
6	El operador le quita la verificación a Elena	Una capacidad, no una insignia
7	Elena intenta iniciar otro live y el servidor la rechaza	El permiso vive en el backend
8	Se restaura la verificación y el live vuelve a funcionar	La decisión es reversible y auditada

El paso 7 es el que sostiene la presentación entera. Conviene ejecutarlo con las herramientas de desarrollo abiertas, o navegando a mano a la ruta bloqueada, para que quede claro que el rechazo no es la interfaz escondiendo un botón.

09

Trampas técnicas conocidas

Cada una cuesta entre media hora y una tarde si se descubre sobre la marcha.

El modelo de auditoría no viaja al cliente

`AuditLog` es `serverOnly`. Hace falta un DTO y un endpoint propios; no alcanza con consultarlo desde Flutter.

Los roles están partidos en dos sistemas

`moderator` y `admin` son scopes del usuario de Serverpod; `creator` vive en el perfil. Una verificación, entonces, es un atributo de perfil y no un scope — mezclarlos obliga a reescribir los guards más adelante.

Auditar fuera de la transacción es auditar mal

`AuditService.record()` acepta una transacción justamente para esto. Registrar después de confirmar deja registros de acciones que fallaron, y registrar antes deja acciones sin registro cuando algo revienta en el medio.

Ocultar no es borrar

Toda acción de moderación debe conservar la evidencia. Un borrado duro deja la cola de reportes apuntando a contenido inexistente, y convierte cualquier apelación posterior en la palabra de uno contra la del otro.

El panel también se desborda a 320 px

Suena absurdo para una consola de escritorio, pero el shell es responsive y el guard de overflow corre sobre toda pantalla montada. Una tabla de auditoría con seis columnas no entra en un teléfono: o se declara como sólo de escritorio, o se diseña la versión angosta.

Documento generado a partir del Notion del proyecto y del estado real del repositorio al 28 de septiembre de 2026. Las capacidades marcadas como "ya existe en mock" están construidas en Flutter y esperan backend.